

DLP - MID - II

RESULTS & FINDINGS

By
SM Anas Imran - K21-3169
DLP-Sec-B

Submitted to
Ms. Sumaiyah Zahid
For the course of Deep Learning For Perception
FAST- National University Of Computer & Emerging Sciences

Task 1 Findings:

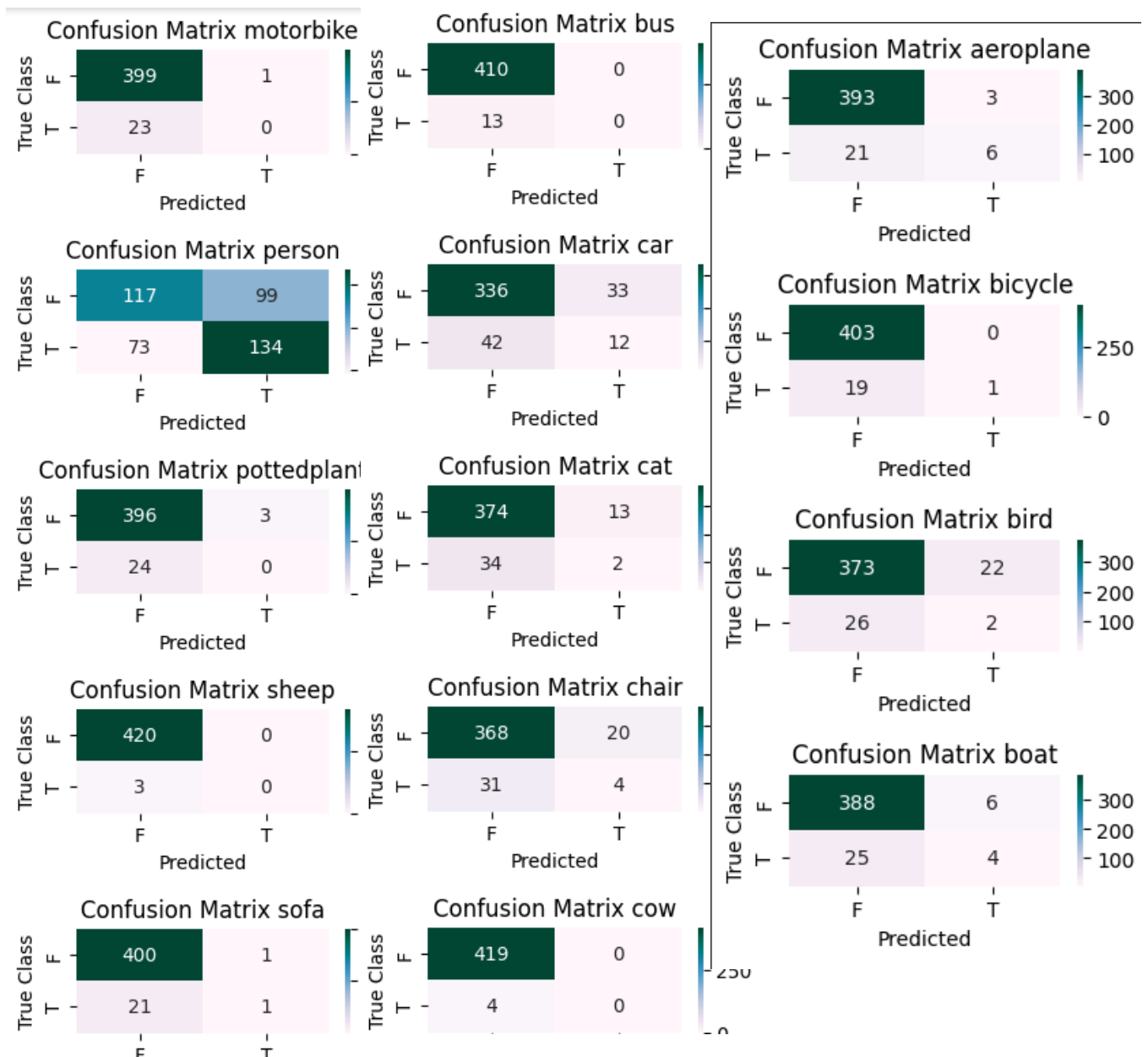
Sequential CNN Architecture

Hyper Params:

Batch Size: 32

Epoch: 10

Confusion Matrices For Some Classes



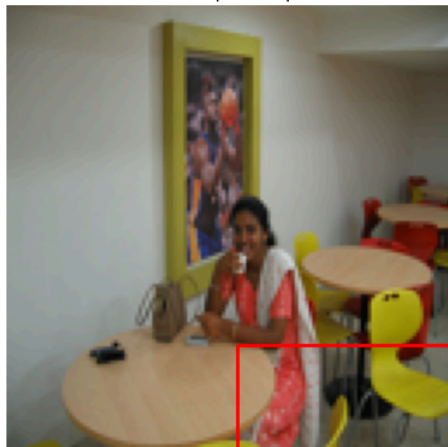
Task 2 Findings

Hyper Params: Batch Size: 64

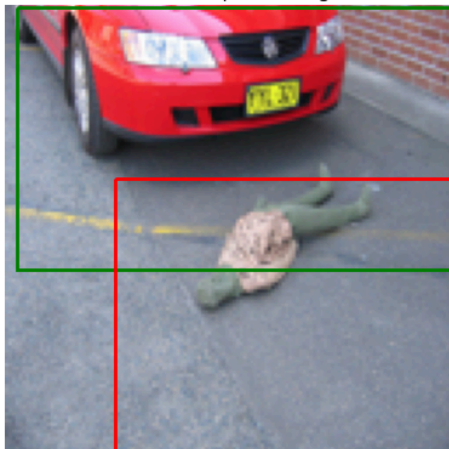
Epoch: 10

Some Samples Below:

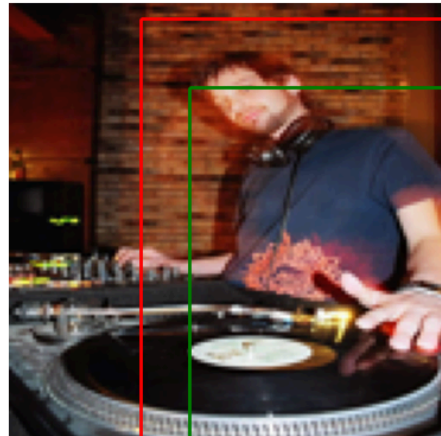
True: chair | Pred: person



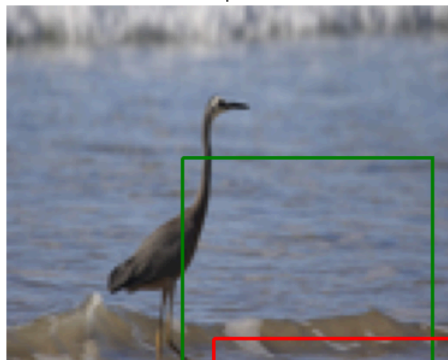
True: car | Pred: dog



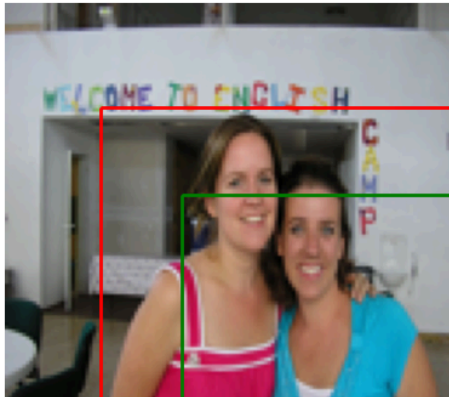
True: person | Pred: person



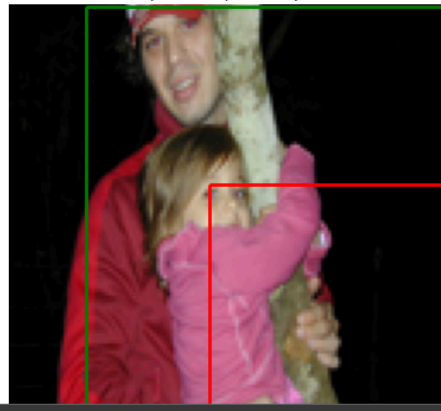
True: bird | Pred: bird



True: person | Pred: person



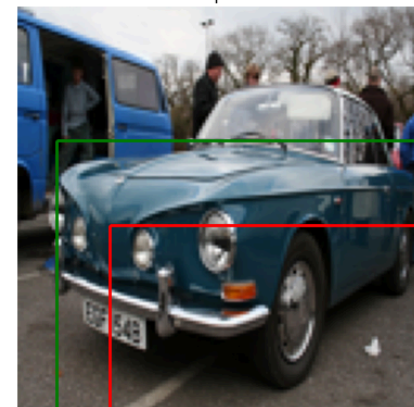
True: person | Pred: person



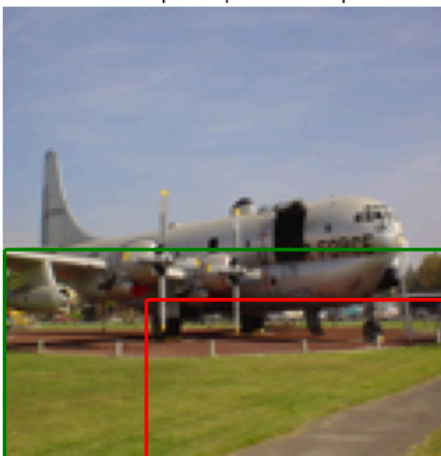
True: aeroplane | Pred: aeroplane



True: car | Pred: car



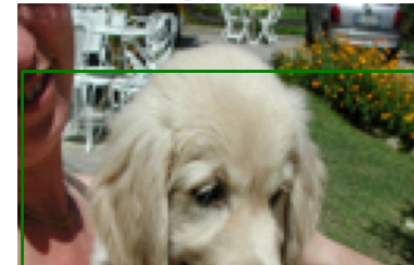
True: aeroplane | Pred: aeroplane



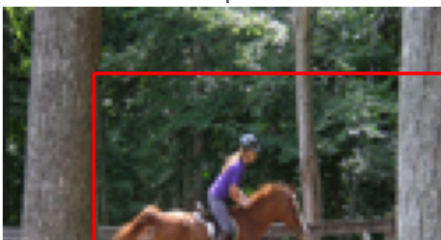
True: aeroplane | Pred: aeroplane



True: dog | Pred: person



True: horse | Pred: horse



Task 3 Findings

Architecture and Hyper Params:

Batch Size: 32

Epoch: 60

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 128, 128, 3)	0	-
conv2d_11 (Conv2D)	(None, 128, 128, 64)	1,792	input_layer_1[0][0]
conv2d_12 (Conv2D)	(None, 128, 128, 64)	36,928	conv2d_11[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 64, 64, 64)	0	conv2d_12[0][0]
conv2d_13 (Conv2D)	(None, 64, 64, 128)	73,856	max_pooling2d_3[0][0]
conv2d_14 (Conv2D)	(None, 64, 64, 128)	147,584	conv2d_13[0][0]
max_pooling2d_4 (MaxPooling2D)	(None, 32, 32, 128)	0	conv2d_14[0][0]
conv2d_15 (Conv2D)	(None, 32, 32, 256)	295,168	max_pooling2d_4[0][0]
conv2d_16 (Conv2D)	(None, 32, 32, 256)	590,880	conv2d_15[0][0]
max_pooling2d_5 (MaxPooling2D)	(None, 16, 16, 256)	0	conv2d_16[0][0]
conv2d_17 (Conv2D)	(None, 16, 16, 512)	1,180,160	max_pooling2d_5[0][0]
conv2d_18 (Conv2D)	(None, 16, 16, 512)	2,359,808	conv2d_17[0][0]
conv2d_transpose_3 (Conv2DTranspose)	(None, 32, 32, 256)	524,544	conv2d_18[0][0]
concatenate_3 (Concatenate)	(None, 32, 32, 512)	0	conv2d_16[0][0], conv2d_transpose_3[0][0]
conv2d_19 (Conv2D)	(None, 32, 32, 256)	1,179,904	concatenate_3[0][0]
conv2d_transpose_4 (Conv2DTranspose)	(None, 64, 64, 128)	131,200	conv2d_19[0][0]
concatenate_4 (Concatenate)	(None, 64, 64, 256)	0	conv2d_14[0][0], conv2d_transpose_4[0][0]
conv2d_20 (Conv2D)	(None, 64, 64, 128)	295,040	concatenate_4[0][0]
conv2d_transpose_5 (Conv2DTranspose)	(None, 128, 128, 64)	32,832	conv2d_20[0][0]
concatenate_5 (Concatenate)	(None, 128, 128, 128)	0	conv2d_12[0][0], conv2d_transpose_5[0][0]
conv2d_21 (Conv2D)	(None, 128, 128, 64)	73,792	concatenate_5[0][0]
conv2d_22 (Conv2D)	(None, 128, 128, 21)	1,365	conv2d_21[0][0]
Total params: 6,924,853 (26.41 MB)			
Trainable params: 6,924,853 (26.41 MB)			

Firstly I trained for 10 and 20 which took some time but resulted in blank blue images with just minor red spots of segmentation. Epoch 20 resulted slight improvement so I tried 60 epochs but that took alot of time,

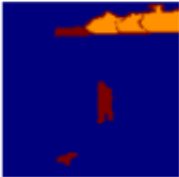
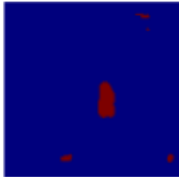
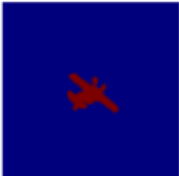
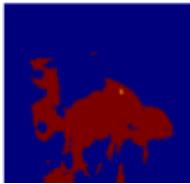
However, I switched my runtime type to T4 , the limited gpu resource provided by colab.. Unfortunately, there was limited gpu provided and I was able to compute the following metrics before the resource was exhausted and my runtime session restarted due to which I was not able to experiment further. Following are and training and its results over 60 epochs

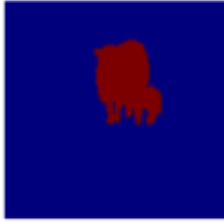
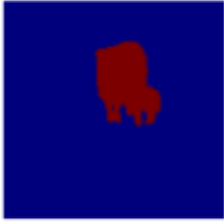
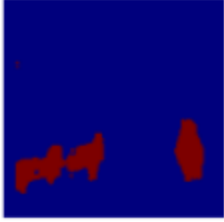
Epoch 1/60	41/41	86s	1s/step	- accuracy: 0.5668	- loss: 1.5000	- val_accuracy: 0.7159	- val_loss: 0.7145
Epoch 2/60	41/41	9s	199ms/step	- accuracy: 0.6983	- loss: 0.6914	- val_accuracy: 0.7255	- val_loss: 0.6267
Epoch 3/60	41/41	8s	203ms/step	- accuracy: 0.7182	- loss: 0.6779	- val_accuracy: 0.7243	- val_loss: 0.6452
Epoch 4/60	41/41	9s	209ms/step	- accuracy: 0.7273	- loss: 0.6418	- val_accuracy: 0.7483	- val_loss: 0.6047
Epoch 5/60	41/41	9s	224ms/step	- accuracy: 0.7392	- loss: 0.6072	- val_accuracy: 0.7614	- val_loss: 0.5857
Epoch 6/60	41/41	10s	206ms/step	- accuracy: 0.7406	- loss: 0.6275	- val_accuracy: 0.7527	- val_loss: 0.6267
Epoch 7/60	41/41	10s	203ms/step	- accuracy: 0.7426	- loss: 0.6127	- val_accuracy: 0.7610	- val_loss: 0.6023
Epoch 8/60	41/41	9s	217ms/step	- accuracy: 0.7478	- loss: 0.5992	- val_accuracy: 0.7545	- val_loss: 0.5815
Epoch 9/60	41/41	10s	215ms/step	- accuracy: 0.7460	- loss: 0.5907	- val_accuracy: 0.7566	- val_loss: 0.5733
Epoch 10/60	41/41	9s	214ms/step	- accuracy: 0.7409	- loss: 0.5961	- val_accuracy: 0.7566	- val_loss: 0.5718
Epoch 11/60	41/41	10s	201ms/step	- accuracy: 0.7580	- loss: 0.5700	- val_accuracy: 0.7592	- val_loss: 0.5689
Epoch 12/60	41/41	8s	204ms/step	- accuracy: 0.7484	- loss: 0.5831	- val_accuracy: 0.7609	- val_loss: 0.5675
Epoch 13/60	41/41	9s	219ms/step	- accuracy: 0.7483	- loss: 0.5859	- val_accuracy: 0.7603	- val_loss: 0.5732
Epoch 14/60	41/41	8s	203ms/step	- accuracy: 0.7509	- loss: 0.5723	- val_accuracy: 0.7576	- val_loss: 0.5693
Epoch 15/60	41/41	9s	221ms/step	- accuracy: 0.7468	- loss: 0.5905	- val_accuracy: 0.7584	- val_loss: 0.5811
Epoch 16/60	41/41	10s	202ms/step	- accuracy: 0.7423	- loss: 0.5882	- val_accuracy: 0.7609	- val_loss: 0.5612
Epoch 17/60	41/41	10s	203ms/step	- accuracy: 0.7468	- loss: 0.5820	- val_accuracy: 0.7605	- val_loss: 0.5637
Epoch 18/60	41/41	11s	215ms/step	- accuracy: 0.7585	- loss: 0.5615	- val_accuracy: 0.7601	- val_loss: 0.5646
Epoch 19/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 20/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 21/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 22/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 23/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 24/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 25/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 26/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 27/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 28/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 29/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 30/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 31/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 32/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 33/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 34/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 35/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 36/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 37/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 38/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 39/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 40/60	41/41	10s	201ms/step	- accuracy: 0.7571	- loss: 0.5691	- val_accuracy: 0.7606	- val_loss: 0.5718
Epoch 41/60	41/41	9s	217ms/step	- accuracy: 0.8173	- loss: 0.4436	- val_accuracy: 0.7819	- val_loss: 0.5396
Epoch 42/60	41/41	10s	203ms/step	- accuracy: 0.8333	- loss: 0.4050	- val_accuracy: 0.7822	- val_loss: 0.5252
Epoch 43/60	41/41	10s	204ms/step	- accuracy: 0.8407	- loss: 0.3834	- val_accuracy: 0.7726	- val_loss: 0.6130
Epoch 44/60	41/41	10s	203ms/step	- accuracy: 0.8274	- loss: 0.4258	- val_accuracy: 0.7815	- val_loss: 0.5484
Epoch 45/60	41/41	8s	202ms/step	- accuracy: 0.8546	- loss: 0.3654	- val_accuracy: 0.7822	- val_loss: 0.5418
Epoch 46/60	41/41	10s	204ms/step	- accuracy: 0.8676	- loss: 0.3350	- val_accuracy: 0.7839	- val_loss: 0.5581
Epoch 47/60	41/41	10s	203ms/step	- accuracy: 0.8844	- loss: 0.2942	- val_accuracy: 0.7837	- val_loss: 0.6198
Epoch 48/60	41/41	10s	204ms/step	- accuracy: 0.8901	- loss: 0.2923	- val_accuracy: 0.7839	- val_loss: 0.5816
Epoch 49/60	41/41	8s	203ms/step	- accuracy: 0.8942	- loss: 0.2746	- val_accuracy: 0.7762	- val_loss: 0.5885
Epoch 50/60	41/41	8s	205ms/step	- accuracy: 0.9067	- loss: 0.2541	- val_accuracy: 0.7821	- val_loss: 0.6322
Epoch 51/60	41/41	8s	204ms/step	- accuracy: 0.9074	- loss: 0.2478	- val_accuracy: 0.7824	- val_loss: 0.6407
Epoch 52/60	41/41	9s	217ms/step	- accuracy: 0.9214	- loss: 0.2119	- val_accuracy: 0.7779	- val_loss: 0.7126
Epoch 53/60	41/41	10s	202ms/step	- accuracy: 0.9307	- loss: 0.1891	- val_accuracy: 0.7818	- val_loss: 0.7300
Epoch 54/60	41/41	11s	215ms/step	- accuracy: 0.9354	- loss: 0.1748	- val_accuracy: 0.7858	- val_loss: 0.7659
Epoch 55/60	41/41	10s	203ms/step	- accuracy: 0.9435	- loss: 0.1540	- val_accuracy: 0.7747	- val_loss: 0.8309
Epoch 56/60	41/41	8s	205ms/step	- accuracy: 0.9532	- loss: 0.1241	- val_accuracy: 0.7791	- val_loss: 0.7856
Epoch 57/60	41/41	10s	204ms/step	- accuracy: 0.9536	- loss: 0.1237	- val_accuracy: 0.7774	- val_loss: 0.8478
Epoch 58/60	41/41	10s	204ms/step	- accuracy: 0.9509	- loss: 0.1269	- val_accuracy: 0.7812	- val_loss: 0.8680
Epoch 59/60	41/41	9s	217ms/step	- accuracy: 0.9625	- loss: 0.0992	- val_accuracy: 0.7697	- val_loss: 0.9052
Epoch 60/60	41/41	8s	204ms/step	- accuracy: 0.9671	- loss: 0.0869	- val_accuracy: 0.7712	- val_loss: 0.9268
Epoch 61/60	41/41	8s	204ms/step	- accuracy: 0.9648	- loss: 0.0932	- val_accuracy: 0.7669	- val_loss: 0.8771

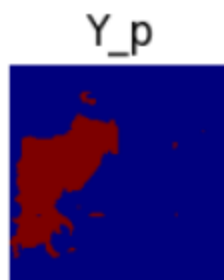
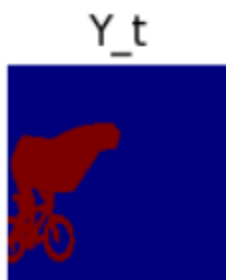
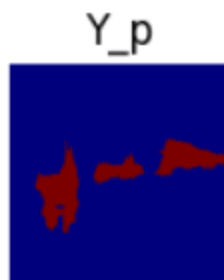
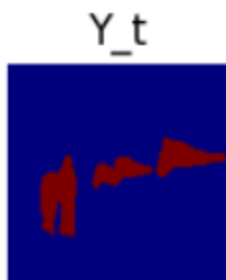
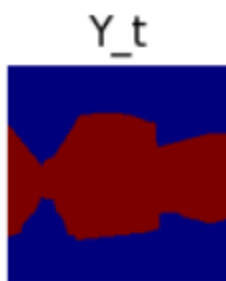
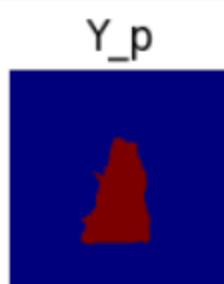
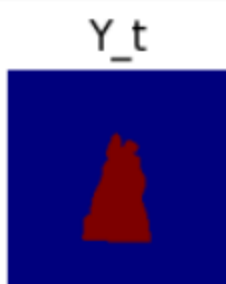
Results:

I : Image Y_t : True Result Y_p : Predicted Segment

 Accuracy: 0.93
Mean IoU: 0.7672440761732279

I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		

I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		
I	Y _t	Y _p
		



Task 4 Findings

Batch Size: 32

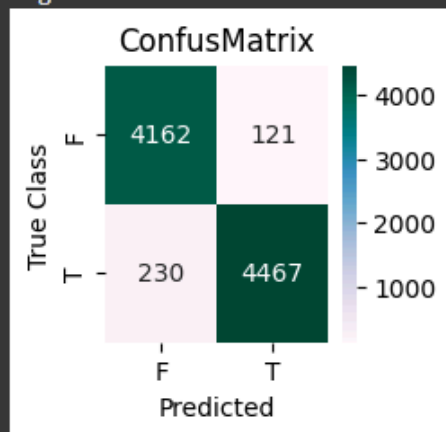
Epoch: 1

I ran RNN first with for 1 epoch and it yielded quite accurate results, Therefore I continued with 1 epoch for all models due to the lack of computational resources, longer training times and good results despite the single epoch.

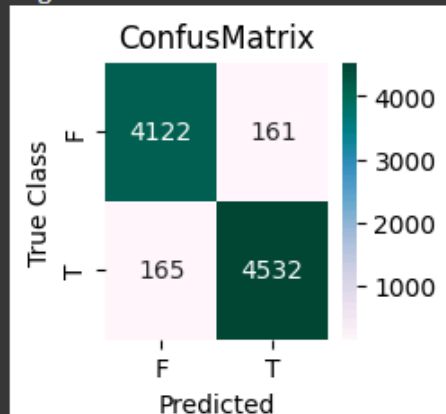
Architecture is been used provided by RNN and LSTM by keras, tensorflow

Results

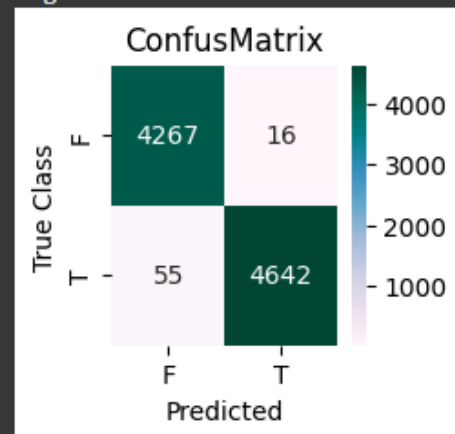
RNN
281/281 ————— 15s 54ms/step
Accuracy : 0.960913140311804
F1 : 0.9621970920840065
Recall : 0.9510325739833937
Precision : 0.9736268526591108
avgAP : 0.9925088697399351



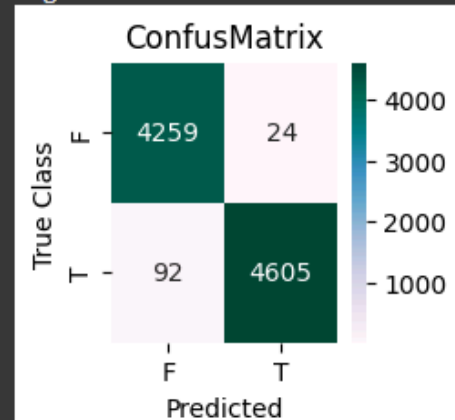
BiRNN
281/281 ————— 27s 93ms/step
Accuracy : 0.9636971046770602
F1 : 0.9652822151224707
Recall : 0.9648711943793911
Precision : 0.9656935861922011
avgAP : 0.9910851450844481



LSTM
281/281 ————— 74s 264ms/step
Accuracy : 0.9920935412026726
F1 : 0.9924104756814538
Recall : 0.9882903981264637
Precision : 0.9965650493774152
avgAP : 0.9988819096912508



Bi-LSTM
281/281 ————— 133s 472ms/step
Accuracy : 0.9870824053452116
F1 : 0.9875616555865323
Recall : 0.9804130295933574
Precision : 0.9948152948801037
avgAP : 0.9982318451536802

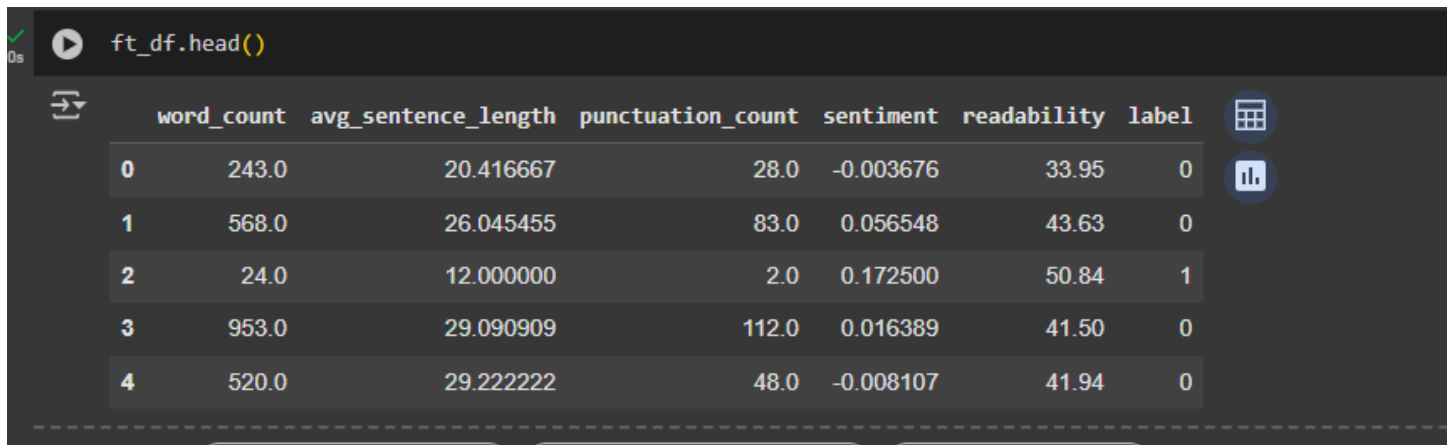


Task 5 Findings

Cleaned the text and extracted features into a dataframe Then applied:

- ML:Random Forest, KNN
- DL Bi RNN, Bi LSTM
-

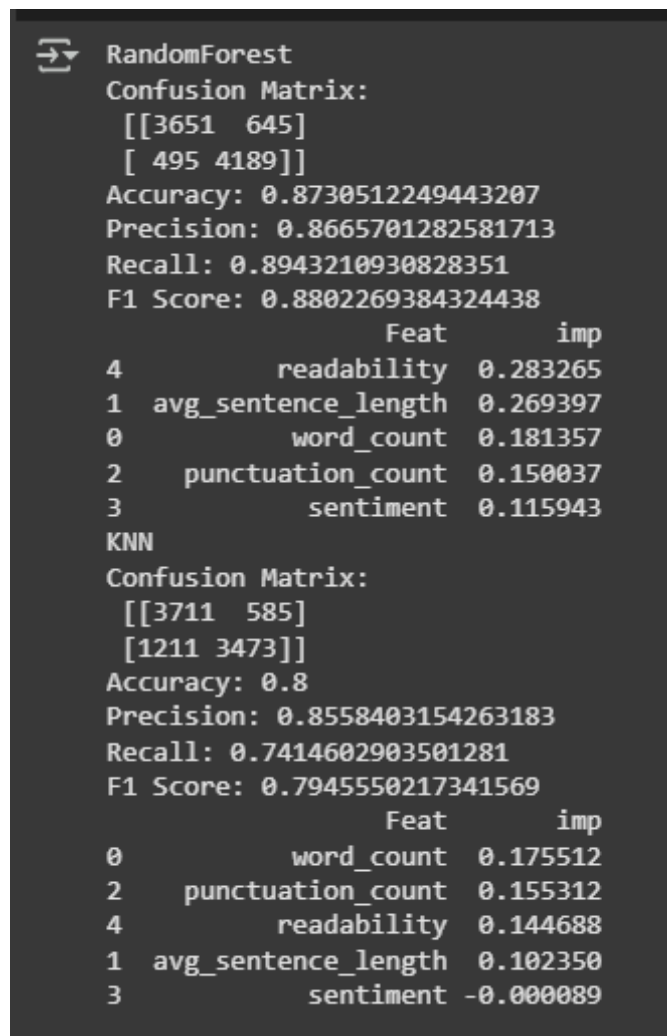
Features Dataframe



The screenshot shows a Jupyter Notebook interface with a code cell containing `ft_df.head()`. Below the code, the first five rows of the dataframe are displayed. The columns are `word_count`, `avg_sentence_length`, `punctuation_count`, `sentiment`, `readability`, and `label`. The rows are indexed 0 through 4.

	word_count	avg_sentence_length	punctuation_count	sentiment	readability	label
0	243.0	20.416667	28.0	-0.003676	33.95	0
1	568.0	26.045455	83.0	0.056548	43.63	0
2	24.0	12.000000	2.0	0.172500	50.84	1
3	953.0	29.090909	112.0	0.016389	41.50	0
4	520.0	29.222222	48.0	-0.008107	41.94	0

ML MODELS:



The screenshot shows a Jupyter Notebook interface with a code cell displaying the results of two machine learning models: Random Forest and KNN. The output includes confusion matrices, accuracy, precision, recall, and F1 scores for both models, followed by a feature importance table for each.

```
RandomForest
Confusion Matrix:
[[3651  645]
 [ 495 4189]]
Accuracy: 0.8730512249443207
Precision: 0.8665701282581713
Recall: 0.8943210930828351
F1 Score: 0.8802269384324438

      Feat      imp
4      readability  0.283265
1  avg_sentence_length  0.269397
0      word_count    0.181357
2  punctuation_count  0.150037
3      sentiment    0.115943

KNN
Confusion Matrix:
[[3711  585]
 [1211 3473]]
Accuracy: 0.8
Precision: 0.8558403154263183
Recall: 0.7414602903501281
F1 Score: 0.7945550217341569

      Feat      imp
0      word_count  0.175512
2  punctuation_count  0.155312
4      readability  0.144688
1  avg_sentence_length  0.102350
3      sentiment -0.000089
```

DL MODELS



Bi-RNN :

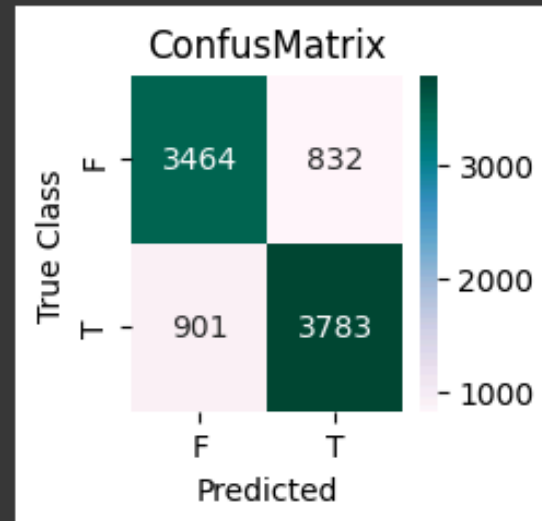
281/281 ————— 1s 3ms/step

Accuracy: 0.8070155902004454

Precision: 0.819718309859155

Recall: 0.8076430401366353

F1 Score: 0.81363587482525



Bi-LSTM :

281/281 ————— 1s 4ms/step

Accuracy: 0.8067928730512249

Precision: 0.8623740476775621

Recall: 0.7491460290350128

F1 Score: 0.8017822460870558

